



DRx LEARNING HUB

Exam-Ready Study Material for Indian University Students

Learn
Practice
Grow

BP101T

Basics of Python Programming

for Pharmaceutical Sciences (Theory)



THE COMPLETE E-BOOK



All 5 Units



Theory



Practice
Programs



MCQs



Long & Short
Answers



Glossary



Model Paper



B.Pharm — Semester I

NEP 2020 / PCI Curriculum

(PCI)



Better Learning
Brighter Future

Compiled & Edited by

DRx Learning Hub



Pharmacy
+
Technology
=
Better Tomorrow

Preface

This e-book has been compiled by DRx Learning Hub to provide B.Pharm Semester I students with a single, complete, exam-ready resource for the course BP101T — Basics of Python Programming for Pharmaceutical Sciences, as prescribed under the NEP 2020 / PCI curriculum.

Python has become an essential skill for the modern pharmacy graduate. From calculating drug dosages and BMI, to cleaning and analysing pharmacokinetic (PK) datasets, to visualising Adverse Drug Reaction (ADR) trends and dissolution profiles — the programming concepts in this course are directly tied to real pharmaceutical and healthcare applications, not taught as abstract computer science.

This complete edition merges all five units of the syllabus — Introduction to Python, Control Structures & Functions, Data Structures & File Handling, Data Handling with Pandas, and Data Visualization with Matplotlib — into one continuous, unit-wise (subject-wise) volume, with long, detailed explanations rather than brief notes. Every concept is explained first in plain language, then reinforced with a working, pharma-flavoured code example, and finally tested through exam-style questions.

The book is organised so that it can be read cover-to-cover as a course, or used unit-wise for quick revision before the Sessional Examination (SE) and End Semester Examination (ESE). Each unit closes with a summary; the book closes with a consolidated glossary, MCQ bank with answer key, fill-in-the-blank practice, short-answer and long-answer question banks, practice/lab programs, and a model question paper built to the official marks distribution.

We hope this e-book makes Python approachable for students coming from a non-computing background, and builds the confidence needed to use data and code as everyday tools in pharmaceutical practice, research, and industry.

— DRx Learning Hub

How to Use This E-Book

- **Read unit-wise:** each of the five units builds on the previous one. Units I–II establish core Python syntax; Units III–V apply it to data handling and visualisation.
- **Boxed notes:** gold-shaded boxes highlight exam tips, common mistakes, and pharma-context notes — do not skip these.
- **Code blocks:** every code block is written to run as-is in Jupyter Notebook, PyCharm, or VS Code. Type them out yourself rather than only reading them.
- **Unit summaries:** use the summary at the end of each unit as a last-minute revision checklist before the exam.
- **Practice sections:** MCQs, fill-in-the-blanks, short-answer and long-answer questions are grouped unit-wise at the end of the book so you can test yourself unit-by-unit.
- **Model paper:** attempt the model question paper under timed conditions before your ESE.

Course Details

Field	Details
Course Code	BP101T
Course Title	Basics of Python Programming for Pharmaceutical Sciences (Theory)
Course Type	Core
Semester	I (First)
Credits	2
Hours per Week (L-T-P)	2 - 0 - 0
Total Hours	30
Maximum Marks	50 (Sessional Exam: 20 + End Semester Exam: 30)

Course Objectives

1. Introduce the fundamentals of Python programming for pharmaceutical sciences.
2. Develop basic programming skills using control structures, functions, and data structures.
3. Provide knowledge of data handling techniques for structured dataset management.
4. Familiarize students with data analysis tools such as NumPy and Pandas for healthcare datasets.
5. Enable students to visualize and interpret pharmaceutical data.

Course Outcomes (CO)

CO No.	Upon completion, students will be able to:
CO1	Explain the fundamentals of Python programming, including variables, data types, operators, and libraries.
CO2	Analyze program logic using control structures and functions.
CO3	Organize, manipulate, and retrieve data using data structures and file handling techniques.
CO4	Analyze pharmaceutical datasets using Python libraries.
CO5	Visualize and interpret pharmaceutical data using graphical tools.

Unit-wise Distribution of Hours

Unit	Title	Hours
I	Introduction to Python Programming	6
II	Control Structures & Functions	6
III	Data Structures & File Handling	6
IV	Data Handling with Pandas	6
V	Data Visualization with Matplotlib	6

Table of Contents

Preface.....	2
How to Use This E-Book.....	3
Course Details.....	4
Unit I — Introduction to Python Programming.....	6
Unit II — Control Structures & Functions.....	12
Unit III — Data Structures & File Handling.....	17
Unit IV — Data Handling with Pandas.....	21
Unit V — Data Visualization with Matplotlib.....	25
Glossary of Key Terms.....	29
Multiple Choice Questions (MCQ Bank).....	31
MCQ Answer Key.....	34
Fill in the Blanks.....	35
Short Answer Questions (2–3 Marks).....	37
Long Answer Questions (10 Marks).....	39
Practice / Lab Programs.....	45
Model Question Paper.....	47
Recommended References.....	48
About DRx Learning Hub.....	48

UNIT I

Introduction to Python Programming

6 Hours

1.1 Why Python for Pharmacy Students?

Python is a high-level, general-purpose, interpreted programming language, created by Guido van Rossum and first released in 1991. It is called a 'high-level' language because it lets you write instructions in a form close to plain English, without worrying about low-level details such as memory addresses. It is called 'interpreted' because a program called the Python interpreter reads and executes your code line-by-line, rather than compiling the entire program into machine code before running it.

For pharmacy students, Python is not taught as computer science for its own sake — it is a practical tool. A B.Pharm graduate today may need to calculate a patient-specific drug dose from body weight, tabulate and clean a spreadsheet of Adverse Drug Reaction (ADR) reports, or plot a drug's concentration-time curve after oral administration. Python performs all of these tasks quickly, reliably, and repeatably, which is exactly what regulatory, clinical, and research environments demand.

Exam Tip

A very common exam question is: 'Why is Python preferred in pharmaceutical data science?' Answer with: free & open-source, easy/readable syntax, huge scientific library ecosystem (NumPy, Pandas, Matplotlib), cross-platform, strong community support, and wide industry adoption in bioinformatics, drug discovery, and clinical data analysis.

1.2 Installing Python

Python can be installed in two common ways, both acceptable for this course:

- **Standard Python (python.org):** download the installer for your operating system (Windows/Mac/Linux) from python.org, run it, and — on Windows — tick the box 'Add Python to PATH' during installation so Python can be run from any command prompt.
- **Anaconda Distribution:** a single installer that bundles Python together with Jupyter Notebook, NumPy, Pandas, and Matplotlib already installed — recommended for data-science-heavy courses like this one, since it saves you from installing each library separately.

After installation, you can verify it from a terminal or command prompt:

```
$ python --version
Python 3.12.3

$ python
>>> print("Hello, Pharmacy!")
Hello, Pharmacy!
```

1.3 Integrated Development Environments (IDEs)

An IDE (Integrated Development Environment) is software that bundles a code editor, a runner/debugger, and helpful features such as auto-completion and error highlighting into a single application. While Python code can technically be written in any plain text editor (Notepad, gedit, etc.) and run from a terminal, an IDE makes writing, testing, and debugging code dramatically faster and less error-prone.

Commonly used IDEs for this course

IDE	Best suited for	Key feature
Jupyter Notebook	Data analysis, step-by-step exploration, visualisation	Cell-based execution — run one block of code at a time and see output immediately below it
PyCharm	Larger, multi-file projects; professional development	Powerful debugger, code refactoring tools, virtual-environment management
VS Code (Visual Studio Code)	Lightweight, flexible, general-purpose coding	Free, extensible via plugins, supports Python via the Python extension

Advantages of IDEs over plain text editors

- Syntax highlighting — keywords, strings, and comments are shown in different colours, making errors easy to spot visually.
- Auto-completion (IntelliSense) — the IDE suggests function/variable names as you type, reducing typing errors.
- Integrated debugger — you can pause execution, inspect variable values, and step through code line-by-line to find bugs.
- Error/warning detection before running — many IDEs underline mistakes as you type, before you even execute the program.
- Integrated terminal and package management — install libraries and run scripts without leaving the IDE.
- In Jupyter specifically: cell-based execution lets you re-run only the changed part of a large analysis instead of the entire script.

Exam Tip

Long-answer question style: 'Differentiate between a text editor and an IDE, and justify why IDEs are recommended for pharmaceutical data programming.' Structure your answer as: Definition of text editor → Definition of IDE → 4-5 point comparison table → conclusion linking to healthcare data work (debugging is critical when processing patient/dosage data, since a silent error could lead to a wrong calculated dose).

1.4 Variables and Data Types

A variable is a named location in a computer's memory used to store a value that can change during program execution. In Python, you do not need to declare the type of a variable in advance (unlike languages such as Java or C) — the type is automatically decided from the value assigned. This feature is called dynamic typing.

```
patient_age = 45          # integer
body_weight = 62.5        # float (decimal number)
drug_name = "Paracetamol" # string
is_diabetic = False       # boolean
```

Python variable names must start with a letter or an underscore, may contain letters, digits, and underscores, are case-sensitive (dose and Dose are different variables), and cannot be a reserved keyword (such as if, for, class).

Core built-in data types

Data type	Description	Example
int	Whole numbers, positive or negative	patient_age = 45
float	Decimal (floating-point) numbers	body_weight = 62.5
str	A sequence of characters (text), written in quotes	drug_name = "Aspirin"
bool	Logical value: True or False	is_pregnant = False
complex	Numbers with a real and imaginary part (rarely used in pharmacy work)	z = 2 + 3j

You can always check a variable's type using the built-in `type()` function, which is frequently asked in exams and vivas:

```
print(type(body_weight)) # <class 'float'>
print(type(drug_name))  # <class 'str'>
```

1.5 Type Casting (Type Conversion)

Type casting is the process of explicitly converting a value from one data type to another using Python's built-in conversion functions: `int()`, `float()`, `str()`, and `bool()`. This is essential in pharmacy programs because data read from user input, files, or web forms usually arrives as text (`str`), even if it represents a number, and must be converted before it can be used in a calculation.

```
age_text = "45"          # this is a string, not a number
age_number = int(age_text) # explicit (manual) type casting to int
print(age_number + 5)     # 50 -- works because it is now an int

height_cm = float("165.5") # string to float
count = str(3)             # int to string, e.g. for concatenation in a message
```

Common Mistake

Trying to add a string and a number directly — e.g. "Dose: " + 500 — raises a `TypeError`. You must first convert the number using `str()`: "Dose: " + str(500). This is one of the most frequently tested 'find the error' style questions.

1.6 Operators

Operators are special symbols that perform operations on values (called operands). Python groups operators into several families relevant to this unit: arithmetic, comparison (relational), and logical.

Arithmetic operators

Operator	Meaning	Example (a=17, b=5)	Result
----------	---------	---------------------	--------

Operator	Meaning	Example (a=17, b=5)	Result
+	Addition	a + b	22
-	Subtraction	a - b	12
*	Multiplication	a * b	85
/	Division (always returns a float)	a / b	3.4
//	Floor division (quotient, rounded down)	a // b	3
%	Modulus (remainder)	a % b	2
**	Exponentiation (power)	a ** 2	289

Comparison (relational) operators

These operators compare two values and always return a Boolean (True or False) result. They are the foundation of decision-making in Unit II.

Operator	Meaning	Example
==	Equal to	dose == 500 → True/False
!=	Not equal to	age != 18
>	Greater than	temp > 37
<	Less than	wbc_count < 4000
>=	Greater than or equal to	age >= 18
<=	Less than or equal to	dose <= max_dose

Logical operators

Logical operators combine multiple Boolean conditions into one — critical when a clinical or dosage decision depends on more than one criterion.

```
age = 70
creatinine_clearance = 45

# and: both conditions must be True
if age > 65 and creatinine_clearance < 50:
    print("Renal dose adjustment required")

# or: at least one condition must be True
# not: reverses a Boolean value
```

1.7 Input and Output Operations

The input() function reads a line typed by the user from the keyboard and always returns it as a string, regardless of what was typed — a fact that is frequently tested in exams. The print() function displays output to the console.

```

name = input("Enter patient name: ")
weight = float(input("Enter body weight (kg): ")) # cast to float immediately
dose_per_kg = 10 # mg/kg
total_dose = weight * dose_per_kg
print("Patient:", name)
print("Recommended dose (mg):", total_dose)

```

Python's `print()` function also supports f-strings (formatted string literals, introduced in Python 3.6), which let you embed variable values directly inside a string using curly braces — the modern, preferred style for building readable output messages:

```
print(f"Patient {name} requires {total_dose} mg of the drug.")
```

1.8 Basic String Operations

A string in Python is an ordered, immutable sequence of characters. 'Immutable' means that once created, the characters of a string cannot be changed in place — any 'modification' actually creates a brand-new string.

Operation	Example	Result
Concatenation (+)	"Para" + "cetamol"	"Paracetamol"
Repetition (*)	"ab" * 3	"ababab"
Length	<code>len("Aspirin")</code>	7
Indexing	<code>"Aspirin"[0]</code>	"A"
Slicing	<code>"Aspirin"[0:3]</code>	"Asp"
Upper/lower case	<code>"aspirin".upper()</code>	"ASPIRIN"
Strip whitespace	<code>" 500mg ".strip()</code>	"500mg"
Replace	<code>"Tab. 500mg".replace("Tab.", "Cap.")</code>	"Cap. 500mg"
Split	<code>"A,B,C".split(",")</code>	['A', 'B', 'C']

Pharma Context

String methods are used constantly when cleaning drug names and batch codes imported from lab or hospital software — e.g. standardising case with `.upper()`, or removing stray spaces from a CSV column with `.strip()` before matching it against a formulary list.

1.9 Standard Libraries and Third-Party Libraries

A library (or module) is a pre-written collection of functions and tools that you can import into your own program instead of writing everything from scratch. Python broadly has two kinds of libraries relevant here:

- **Standard libraries:** ship with Python itself and need no separate installation, e.g. `math` (mathematical functions), `datetime` (dates and times), `csv` (reading/writing CSV files), `os` (operating-system interaction).
- **Third-party libraries:** written by the wider Python community and must be installed separately, e.g. `numpy`, `pandas`, `matplotlib` — the libraries central to Units III, IV, and V of this course.

```
import math
print(math.sqrt(144))    # 12.0  -- standard library, no installation needed

import numpy as np       # third-party library, must be installed first
```

Installing and uninstalling third-party libraries with pip

pip (Pip Installs Packages) is Python's standard package manager. It downloads a library from the Python Package Index (PyPI) and installs it into your Python environment.

```
pip install pandas        # installs the Pandas library
pip install numpy matplotlib # multiple libraries in one command
pip uninstall pandas      # removes an installed library
pip show pandas           # displays version and details of an installed library
pip list                  # lists all installed libraries
```

Exam Tip

A frequent one-mark/short question: 'What is the difference between a standard library and a third-party library, and what command is used to install the latter?' Answer: standard libraries are bundled with Python and need no installation; third-party libraries (e.g. NumPy, Pandas) must be installed using pip install <library-name>.

Unit I — Summary

Quick Revision

Python is installed from python.org or via Anaconda; IDEs (Jupyter, PyCharm, VS Code) add syntax highlighting, auto-completion, and debugging over plain text editors; variables are dynamically typed and hold int/float/str/bool values; type casting converts between types with int()/float()/str()/bool(); arithmetic, comparison, and logical operators manipulate and compare values; input() always returns a string and print()/f-strings produce output; strings support indexing, slicing, and methods like .upper()/strip()/split()/replace(); standard libraries need no installation while third-party libraries (NumPy, Pandas, Matplotlib) are installed via pip install.

UNIT II**Control Structures & Functions***6 Hours***2.1 Conditional Statements**

By default, Python executes statements sequentially, one after another, from top to bottom. Conditional statements allow a program to make decisions and execute different blocks of code depending on whether a condition is True or False — essential whenever a pharmacy program must respond differently for different patients, e.g. adjusting a dose only if renal function is impaired.

Python uses indentation (whitespace at the start of a line, conventionally 4 spaces) — not curly braces {} as in C/Java — to define which statements belong inside a block. Incorrect indentation is one of the most common sources of errors for beginners and a favourite 'spot the error' exam question.

if statement

```
temperature = 39.2
if temperature > 37.5:
    print("Patient has fever")
```

if-else statement

```
age = 15
if age >= 18:
    print("Adult dose applies")
else:
    print("Paediatric dose applies")
```

if-elif-else (nested/multi-way) statement

elif (short for 'else if') lets a program check several mutually-exclusive conditions in sequence, stopping at the first one that is True.

```
bmi = 27.4
if bmi < 18.5:
    category = "Underweight"
elif bmi < 25:
    category = "Normal"
elif bmi < 30:
    category = "Overweight"
else:
    category = "Obese"
print("BMI Category:", category)
```

Nested conditions

An if statement placed inside another if (or elif/else) block is called a nested condition, used when a decision itself depends on more than one layered criterion.

```
age = 72
creatinine = 1.8
if age > 65:
    if creatinine > 1.5:
        print("High risk: reduce dose and monitor renal function")
    else:
        print("Elderly, but renal function acceptable")
else:
    print("Standard adult dosing")
```

Exam Tip

Long-answer style: 'Write a Python program to categorise a patient's BMI and explain the logic used.' Marks are awarded for: correct if-elif-else structure, correct WHO BMI cut-offs (18.5, 25, 30), taking input via input()/float(), and a clear final print() statement — examiners specifically check that every elif condition is written relative to the previous one (e.g. `bmi < 25`, not `18.5 <= bmi < 25`, since `elif` already assumes the prior condition failed).

2.2 Loops

A loop repeats a block of code multiple times, avoiding the need to write the same instructions repeatedly. Python provides two loop constructs.

for loop

A for loop iterates over a sequence (a list, string, or a range of numbers), executing the loop body once for every item in that sequence. It is used when the number of repetitions is known or is tied to a collection of items.

```
doses_mg = [250, 500, 750, 1000]
for dose in doses_mg:
    print(f"Dose option: {dose} mg")

# range(start, stop, step) generates a sequence of numbers
for day in range(1, 8):          # 1 to 7 (stop value is excluded)
    print(f"Day {day}: take tablet")
```

while loop

A while loop repeats a block of code as long as a given condition remains True. It is used when the number of repetitions is not known in advance and depends on a condition being met (e.g. keep dispensing until stock runs out).

```
stock = 50
dispensed = 0
while stock > 0:
    stock -= 10          # dispense 10 units
    dispensed += 1
print(f"Dispensed {dispensed} times; stock now {stock}")
```

Common Mistake

Forgetting to update the loop-control variable inside a while loop (e.g. forgetting `stock -= 10`) creates an infinite loop that never stops — a classic exam 'identify the bug' question.

2.3 break, continue, and pass Statements

Statement	Effect	Typical use
<code>break</code>	Immediately exits the loop entirely, skipping any remaining iterations	Stop searching a batch list once the required batch number is found
<code>continue</code>	Skips the rest of the current iteration and moves to the next one	Skip a record with a missing/invalid dosage value while still processing the rest
<code>pass</code>	Does nothing — a placeholder used where a statement is syntactically required	Leaving a function or if-block empty temporarily during program planning

```
batch_numbers = ["A101", "A102", "A103", "A104"]
target = "A103"
for b in batch_numbers:
    if b == target:
        print("Batch found:", b)
        break

readings = [98, -1, 102, 97, -1, 100]  # -1 marks a missing reading
for r in readings:
    if r == -1:
        continue  # skip invalid reading, do not process it
    print("Valid reading:", r)
```

2.4 Functions

A function is a named, reusable block of code that performs a specific task. Functions let a large program be broken into small, testable, and reusable pieces — instead of copying the same dosage-calculation code everywhere it is needed, it is written once inside a function and simply called wherever required.

Defining and calling a function

```
def calculate_dose(weight_kg, dose_per_kg):    """Return the total dose in mg for a
given body weight."""
    total = weight_kg * dose_per_kg
    return total

# calling (invoking) the function with arguments
patient_dose = calculate_dose(62.5, 10)
print(f"Total dose: {patient_dose} mg")
```

`def` is the keyword used to define a function. The values passed into the function (`weight_kg`, `dose_per_kg` above) are called parameters when listed in the definition, and arguments when supplied at the call. The return statement sends a value back to the caller and immediately ends the function's execution; a function with no return statement automatically returns the special value `None`.

Parameters, default arguments, and keyword arguments

```
def calculate_dose(weight_kg, dose_per_kg=10):    # default argument
    return weight_kg * dose_per_kg

print(calculate_dose(70))                      # uses default dose_per_kg = 10
print(calculate_dose(70, dose_per_kg=15))# keyword argument, overrides default
```

Exam Tip

Know the vocabulary precisely: 'parameter' = the name in the function definition; 'argument' = the actual value supplied when calling the function. Examiners frequently ask you to differentiate these two terms directly.

2.5 Writing Modular Programs for Pharmaceutical Applications

A 'modular program' breaks a task into small functions, each responsible for one clearly defined job. The syllabus specifically names two worked applications for this unit: dosage calculation and BMI calculation. Both are commonly asked as full long-answer programming questions.

Application 1 — Dosage Calculation Program

```
def calculate_dose(weight_kg, dose_per_kg):
    """Calculate total drug dose (mg) from body weight and dose/kg."""
    return weight_kg * dose_per_kg

def classify_patient(age):
    """Return 'Paediatric', 'Adult', or 'Geriatric' based on age."""
    if age < 12:
        return "Paediatric"
    elif age < 65:
        return "Adult"
    else:
        return "Geriatric"

def main():
    name = input("Enter patient name: ")
    age = int(input("Enter age (years): "))
    weight = float(input("Enter weight (kg): "))
    dose_per_kg = float(input("Enter prescribed dose (mg/kg): "))

    category = classify_patient(age)
    dose = calculate_dose(weight, dose_per_kg)

    print(f"\nPatient: {name}")
    print(f"Category: {category}")
    print(f"Total dose required: {dose} mg")

main()
```

Application 2 — BMI Calculation Program

BMI (Body Mass Index) is calculated as weight in kilograms divided by the square of height in metres: $\text{BMI} = \text{weight (kg)} / [\text{height (m)}]^2$.

```
def calculate_bmi(weight_kg, height_m):
    """Return BMI given weight in kg and height in metres."""
    return weight_kg / (height_m ** 2)

def classify_bmi(bmi):
    """Return the WHO BMI category as a string."""
    if bmi < 18.5:
        return "Underweight"
    elif bmi < 25:
        return "Normal weight"
    elif bmi < 30:
        return "Overweight"
    else:
        return "Obese"

def main():
    weight = float(input("Enter weight (kg): "))
    height = float(input("Enter height (m): "))
    bmi = round(calculate_bmi(weight, height), 2)
    category = classify_bmi(bmi)
    print(f"BMI: {bmi} -> {category}")

main()
```

Exam Tip

When answering a 'write a program' long question, always: (1) define clear, single-purpose functions with docstrings, (2) cast input() values with float()/int() immediately, (3) include a main() driver function that ties everything together, and (4) round() decimal results for readable output. Examiners award separate marks for correct function structure, correct logic, and correct input/output handling.

Unit II — Summary

Quick Revision

Conditional statements (if / if-else / if-elif-else, including nested conditions) let a program branch its logic based on Boolean tests; indentation defines code blocks in Python. for loops iterate over sequences/ranges; while loops repeat while a condition holds. break exits a loop early, continue skips to the next iteration, and pass is a no-op placeholder. Functions (def ... return) package reusable logic; parameters are the names in the definition, arguments are the values passed at the call; default and keyword arguments add flexibility. Modular programs combine several small functions with a main() driver — demonstrated here through dosage-calculation and BMI-calculation applications.

UNIT III

Data Structures & File Handling

6 Hours

3.1 Lists, Tuples, and Dictionaries

A data structure is a way of organising and storing a collection of related values under a single variable name, rather than creating a separate variable for each individual value. Python provides several built-in data structures; three are central to this unit.

Lists

A list is an ordered, mutable (changeable) collection of items, written inside square brackets []. Lists may contain items of mixed types and may contain duplicate values.

```
drug_names = ["Paracetamol", "Ibuprofen", "Amoxicillin"]
batch_quantities = [120, 85, 60]

drug_names.append("Cetirizine")      # add an item at the end
drug_names.remove("Ibuprofen")      # remove a specific item
drug_names[0] = "Paracetamol 500mg" # lists are mutable -- items can be reassigned
print(len(drug_names))               # number of items
```

Tuples

A tuple is an ordered, immutable collection, written inside round brackets (). Once created, a tuple's contents cannot be changed — making tuples ideal for data that must not be accidentally altered, such as a fixed set of reference values (e.g. normal blood-pressure range).

```
normal_bp_range = (90, 120) # (systolic_low, systolic_high) -- fixed reference values
# normal_bp_range[0] = 100 # this would raise a TypeError -- tuples cannot be modified
```

Dictionaries

A dictionary stores data as key-value pairs, written inside curly braces { }. Instead of accessing items by numeric position (as in lists), items are accessed by a unique, meaningful key — very well suited to storing a patient's or drug's structured record.

```
patient_record = {
    "name": "Rahul Verma",
    "age": 34,
    "weight_kg": 68.5,
    "allergies": ["Penicillin"]
}
print(patient_record["age"])      # access value by key -> 34
patient_record["age"] = 35        # update a value
patient_record["diagnosis"] = "Type 2 Diabetes" # add a new key-value pair
```

Feature	List []	Tuple ()	Dictionary { }
Ordered	Yes	Yes	Yes (insertion order, Python 3.7+)
Mutable (changeable)	Yes	No	Yes
Accessed by	Index (position)	Index (position)	Key (name)
Allows duplicates	Yes	Yes	Keys must be unique
Typical pharmacy use	List of drug names / batch numbers	Fixed reference ranges, coordinates	A single patient's or drug's structured record

Exam Tip

A frequent long-answer question: 'Differentiate between list, tuple, and dictionary with suitable pharmaceutical examples.' Structure using the table above, then give one clear example for each data structure.

3.2 Indexing and Slicing

Indexing accesses a single element of a sequence (list, tuple, or string) by its position number. Python indexing starts at 0 for the first element, and negative indices count backwards from the end (-1 is the last element).

```
drugs = ["Paracetamol", "Ibuprofen", "Amoxicillin", "Cetirizine"]
print(drugs[0])      # "Paracetamol" (first element)
print(drugs[-1])     # "Cetirizine" (last element)
```

Slicing extracts a sub-portion of a sequence using the syntax sequence[start:stop:step]. The start index is included; the stop index is excluded; step (optional) controls the interval between picked elements.

```
print(drugs[1:3])     # ['Ibuprofen', 'Amoxicillin'] -- index 1 up to (not including) 3
print(drugs[:2])      # ['Paracetamol', 'Ibuprofen'] -- from start up to index 2
print(drugs[::2])     # ['Paracetamol', 'Amoxicillin'] -- every 2nd element
print(drugs[::-1])    # reversed list
```

Basic operations on lists and dictionaries

Operation	List example	Dictionary example
Add item	lst.append(x)	d[new_key] = value
Remove item	lst.remove(x) / lst.pop(i)	del d[key]
Check membership	x in lst	key in d
Length	len(lst)	len(d)
Iterate	for x in lst:	for k, v in d.items():
Sort	lst.sort()	sorted(d.items())

3.3 String Manipulation Techniques (Revisited)

Building on Unit I, exam questions in this unit often combine string methods with lists — for example, splitting a raw text line from a lab report into a list of separate fields:

```
raw_line = "A101, Paracetamol, 500mg, 120"
fields = [f.strip() for f in raw_line.split(",")]
print(fields) # ['A101', 'Paracetamol', '500mg', '120']
```

3.4 Introduction to NumPy Arrays

NumPy (Numerical Python) is a third-party library that provides the ndarray (n-dimensional array) — a fast, memory-efficient way to store and compute over large collections of numbers, and the foundation on which Pandas (Unit IV) is built.

Why NumPy arrays instead of plain Python lists?

- Arithmetic on a NumPy array applies to every element at once (vectorised operations), without writing a loop.
- NumPy arrays use less memory and run mathematical operations much faster than equivalent Python lists, important when processing large clinical datasets.
- NumPy provides ready-made statistical functions (mean, std, min, max, sum) used constantly in pharmaceutical data analysis.

```
import numpy as np

dissolution_times = np.array([5, 10, 15, 20, 30, 45, 60]) # array creation
percent_dissolved = np.array([12.5, 28.0, 41.2, 55.8, 74.1, 89.6, 96.3])

print(percent_dissolved * 1)      # element-wise operation
print(percent_dissolved.mean())   # average % dissolved
print(percent_dissolved.max())    # maximum value
print(dissolution_times + 5)      # arithmetic on the whole array at once
```

Pharma Context

NumPy arrays are the natural data type for a dissolution profile (% drug released vs time) or a set of repeated assay readings, since every value can be scaled, averaged, or compared in one line of vectorised code instead of a manual loop.

3.5 Reading and Writing CSV Files

CSV (Comma-Separated Values) is the most common plain-text format for tabular data — every line is one row, and commas separate the columns. Hospital and lab information systems routinely export patient, prescription, and assay data as CSV. Python's built-in csv module (a standard library — no installation needed) reads and writes these files.

```
import csv

# Reading a CSV file
with open("patients.csv", "r") as file:
    reader = csv.reader(file)
```

```

header = next(reader)          # first row = column names
for row in reader:
    print(row)

# Writing a CSV file
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age", "Dose_mg"]) # header row
    writer.writerow(["Rahul", 34, 650])

```

The `with open(...)` as ... syntax is called a context manager: it automatically closes the file once the indented block finishes, even if an error occurs — the recommended and examiner-preferred way to handle files in Python, over the older `open()/close()` pattern.

3.6 Structured Healthcare Datasets

A 'structured' dataset is one organised into a fixed set of named columns (fields) and rows (records) — exactly the shape of a CSV or spreadsheet file. Typical structured healthcare datasets encountered in pharmaceutical data work include: patient demographic records (Patient ID, Age, Sex, Weight), prescription/dispensing logs (Drug, Dose, Date, Batch No.), ADR report registers (Drug, Reaction, Severity, Outcome), and PK study datasets (Subject ID, Time, Plasma Concentration).

Understanding a dataset's structure — which columns exist, their data types, and whether values are missing — is the essential first step before any analysis, and is the direct link to the Pandas techniques covered in Unit IV.

3.7 Importing and Manipulating Small Pharmaceutical Datasets

```

import csv

with open("adr_reports.csv", "r") as file:
    reader = csv.DictReader(file) # reads each row as a dictionary using the header as keys
    severe_reactions = []
    for row in reader:
        if row["Severity"] == "Severe":
            severe_reactions.append(row["Drug"])

print("Drugs with severe ADRs:", severe_reactions)

```

Exam Tip

csv.DictReader is a favourite exam topic because it combines two units at once: file handling (Unit III) and dictionaries (Unit III). Remember it reads each row as an ordered dictionary keyed by the column headers, so fields are accessed by name (row["Severity"]) rather than by numeric index.

Unit III — Summary

Quick Revision

Lists [] are ordered & mutable; tuples () are ordered & immutable; dictionaries { } store key-value pairs accessed

by key. Indexing (0-based, negative from the end) picks a single item; slicing [start:stop:step] extracts a sub-sequence. NumPy's ndarray supports fast, vectorised numeric operations (mean(), max(), element-wise arithmetic) and underlies Pandas. The csv module (standard library) reads/writes CSV files using csv.reader/csv.writer/csv.DictReader inside a with open(...) as ... block, which auto-closes the file. Structured healthcare datasets (patient records, ADR logs, PK datasets) are organised into named columns and rows, ready for the Pandas-based analysis covered in Unit IV.

UNIT IV

Data Handling with Pandas

6 Hours

4.1 Introduction to the Pandas Library

Pandas is a third-party Python library built on top of NumPy, purpose-built for working with structured (tabular) data — exactly the shape of the healthcare datasets introduced in Unit III. Where NumPy is optimised for numeric arrays, Pandas adds labelled rows and columns, mixed data types per column, and powerful tools for cleaning, filtering, and summarising real-world, imperfect data — including datasets with missing values, which are extremely common in clinical and laboratory records.

```
pip install pandas      # install once, if not already available (e.g. via Anaconda)
import pandas as pd     # 'pd' is the universal, conventional alias for pandas
```

4.2 Pandas Series and DataFrame Structures

Series

A Series is a one-dimensional, labelled array — essentially a single column of data, where each value has an associated index label.

```
import pandas as pd
ages = pd.Series([45, 62, 34, 58], name="Age")
print(ages)
print(ages.mean())      # 49.75
```

DataFrame

A DataFrame is a two-dimensional, labelled table — rows and columns, exactly like a spreadsheet — and is the primary structure used throughout this unit. Each column of a DataFrame is itself a Series.

```
data = {
    "PatientID": ["P001", "P002", "P003", "P004"],
    "Age": [45, 62, 34, 58],
    "Weight_kg": [70.5, 68.2, None, 75.0],    # None -- a missing value
    "Drug": ["Metformin", "Atenolol", "Metformin", "Losartan"]
}
df = pd.DataFrame(data)
print(df)
```

Exam Tip

A very commonly asked question: 'Differentiate between a Pandas Series and a DataFrame.' Answer: a Series is one-dimensional (a single labelled column); a DataFrame is two-dimensional (multiple columns/rows, like a table); every column inside a DataFrame is itself a Series.

4.3 Reading CSV and Excel Files

Pandas provides direct, one-line functions to load an entire spreadsheet or CSV file into a DataFrame — far simpler than the manual csv module approach from Unit III, and the standard method used for PK study datasets and ADR reports in real pharmaceutical practice.

```
pk_data = pd.read_csv("pk_study.csv")          # read a CSV file
adr_data = pd.read_excel("adr_reports.xlsx", sheet_name="2025") # read an Excel file

pk_data.to_csv("pk_cleaned.csv", index=False) # write a DataFrame back out to CSV
```

Common Mistake

Forgetting index=False when writing back to CSV creates an extra, unwanted 'Unnamed: 0' column in the saved file, containing Pandas' internal row index — always specified as a to_csv() best practice.

4.4 Inspecting Datasets: head(), tail(), info(), describe()

Function	Purpose
df.head(n)	Shows the first n rows (default 5) — a quick first look at the data
df.tail(n)	Shows the last n rows (default 5)
df.info()	Shows column names, data types, and count of non-null values per column
df.describe()	Shows summary statistics (count, mean, std, min, 25%, 50%, 75%, max) for numeric columns
df.shape	Returns (number_of_rows, number_of_columns) as a tuple
df.columns	Lists all column names

```
print(pk_data.head())      # first 5 rows
print(pk_data.info())      # column types & missing-value counts
print(pk_data.describe())  # mean, min, max, quartiles, etc.
```

4.5 Data Cleaning and Handling Missing Values

Real pharmaceutical and clinical datasets almost always contain missing, blank, or inconsistent values — a patient's weight not recorded, a reading skipped, or a batch number mistyped. Pandas represents a missing value as NaN (Not a Number) and provides dedicated tools to detect and handle it.

```
print(df.isnull())          # True/False grid marking missing values
print(df.isnull().sum())    # count of missing values per column

df_dropped = df.dropna()    # remove rows with ANY missing value
df_filled = df.fillna(0)    # replace missing values with 0
df["Weight_kg"] = df["Weight_kg"].fillna(df["Weight_kg"].mean()) # fill with column mean
```

Pharma Context

Which strategy to use matters clinically: `dropna()` is appropriate when a record is unusable without that field (e.g. a PK sample with no recorded time-point); filling with the column mean/median is a reasonable estimate for a physiological measurement like weight, but should never be used to fabricate a missing efficacy or safety outcome.

4.6 Filtering and Selecting Data

Selecting specific columns, or filtering rows based on a condition, is one of the most exam-relevant Pandas skills, since it directly parallels how a pharmacist or researcher isolates a sub-group of patients or a specific drug's records.

```
print(df["Age"])                # select a single column (returns a Series)
print(df[["PatientID", "Age"]]) # select multiple columns (returns a DataFrame)

elderly = df[df["Age"] > 60]      # filter rows: only patients older than 60
metformin_pts = df[df["Drug"] == "Metformin"] # filter rows by drug name

# combining conditions with & (and) / | (or) -- note the brackets around each condition
elderly_metformin = df[(df["Age"] > 60) & (df["Drug"] == "Metformin")]

print(df.loc[0, "Drug"]) # label-based access: row label 0, column 'Drug'
print(df.iloc[0, 3])     # position-based access: row 0, column index 3
```

Exam Tip

Know the distinction: `.loc[]` selects by label/name (row/column names), while `.iloc[]` selects by integer position — a frequent short-answer/MCQ pairing.

4.7 Grouping Data and Aggregation Functions

`groupby()` splits a DataFrame into groups based on the unique values of one or more columns, so that a summary statistic can be calculated separately for each group — for example, the average dose given per drug, or the number of ADRs reported per severity level.

```
avg_age_by_drug = df.groupby("Drug")["Age"].mean()
print(avg_age_by_drug)

adr_counts = adr_data.groupby("Severity")["Drug"].count()
print(adr_counts)

summary = df.groupby("Drug").agg({"Age": "mean", "Weight_kg": "max"}) # multiple
aggregations at once
```

Aggregation function	Meaning
<code>mean()</code>	Average value
<code>sum()</code>	Total
<code>count()</code>	Number of (non-null) entries

Aggregation function	Meaning
min() / max()	Smallest / largest value
std()	Standard deviation

Unit IV — Summary

Quick Revision

Pandas (import pandas as pd) provides Series (1-D labelled column) and DataFrame (2-D labelled table) structures. pd.read_csv() / pd.read_excel() load data; df.to_csv() saves it. df.head()/tail()/info()/describe() give a quick inspection of any new dataset. Missing values (NaN) are found with isnull().sum() and handled with dropna() or fillna(). Rows are filtered with Boolean conditions (df[df["col"] > x]), columns selected with df[[]], and .loc[]/.iloc[] give label-based / position-based access. groupby() followed by an aggregation function (mean(), sum(), count()) summarises data by category — the core technique for analysing pharmaceutical datasets by drug, severity, or patient group.

UNIT V

Data Visualization with Matplotlib

6 Hours

5.1 Introduction to Matplotlib

Matplotlib is Python's foundational third-party plotting library, used to turn numeric data — the output of the NumPy and Pandas work in Units III and IV — into visual charts and graphs. In pharmaceutical science, a well-chosen graph often communicates a finding (a concentration-time profile, a dissolution curve, an ADR trend) far more clearly than a table of numbers, which is precisely why data visualisation is the final, capstone unit of this course.

```
pip install matplotlib          # install once, if needed
import matplotlib.pyplot as plt # 'plt' is the conventional, universal alias
```

The pyplot module (plt) provides a simple, MATLAB-style interface: you build a figure step-by-step using function calls, then display it with plt.show().

5.2 Creating Basic Plot Types

Line plots

A line plot connects data points with a line and is the standard choice for data measured over a continuous variable such as time — making it the natural chart for a concentration-time (PK) curve.

```
import matplotlib.pyplot as plt

time_hr = [0, 0.5, 1, 2, 4, 6, 8, 12]
conc_ng_ml = [0, 45, 78, 92, 65, 40, 22, 8]

plt.plot(time_hr, conc_ng_ml, marker="o", color="green")
plt.show()
```

Histograms

A histogram groups continuous numeric data into intervals (bins) and shows how many values fall into each bin — used to see the distribution/spread of a variable, e.g. the spread of patient ages or body-weight values in a study population.

```
ages = [22, 25, 30, 31, 35, 40, 41, 45, 50, 55, 60, 61, 65, 70]
plt.hist(ages, bins=6, color="gold", edgecolor="black")
plt.show()
```

Scatter plots

A scatter plot places each pair of (x, y) values as an individual point, used to examine the relationship or correlation between two numeric variables — e.g. dose given versus response observed.

```
dose_mg = [50, 100, 150, 200, 250, 300]
response_pct = [10, 22, 38, 55, 68, 80]
plt.scatter(dose_mg, response_pct, color="darkgreen")
plt.show()
```

Box plots

A box plot summarises a variable's distribution using five key statistics (minimum, first quartile, median, third quartile, maximum) and clearly flags outliers — very useful for comparing, e.g., dissolution results across several tablet batches at a glance.

```
batch_a = [92, 94, 95, 96, 97, 99, 85]
batch_b = [88, 90, 91, 93, 94, 95, 78]
plt.boxplot([batch_a, batch_b], labels=["Batch A", "Batch B"])
plt.show()
```

Plot type	Function	Best for
Line plot	plt.plot()	Trends over a continuous variable, e.g. concentration vs time
Histogram	plt.hist()	Distribution/spread of a single numeric variable
Scatter plot	plt.scatter()	Relationship between two numeric variables
Box plot	plt.boxplot()	Comparing spread/outliers across groups or batches

5.3 Labeling Axes, Titles, and Legends

A chart without labels is scientifically incomplete and will lose marks in a practical/viva examination — every plot must clearly state what is being measured and in what units.

```
plt.plot(time_hr, conc_ng_ml, marker="o", color="green", label="Oral dose")
plt.title("Plasma Concentration vs Time")
plt.xlabel("Time (hours)")
plt.ylabel("Plasma Concentration (ng/mL)")
plt.legend()          # displays the label(s) defined above, as a key
plt.grid(True)        # optional -- adds a background grid for easier reading
plt.show()
```

Exam Tip

A common short-answer question: 'List the essential components of a properly labelled Matplotlib graph.' Answer: plt.title() for the chart title, plt.xlabel()/plt.ylabel() for axis labels (with correct units), plt.legend() to identify multiple data series, and appropriately chosen plt.plot()/plt.hist()/plt.scatter()/plt.boxplot() for the data type being shown.

5.4 Visualizing Pharmaceutical Datasets

The syllabus specifically names three worked pharmaceutical visualisation scenarios, all of which are common long-answer/practical exam questions.

(a) Concentration-time curves for oral and IV administration

```
time_hr = [0, 0.5, 1, 2, 4, 6, 8, 12]
oral_conc = [0, 45, 78, 92, 65, 40, 22, 8]
iv_conc = [150, 130, 110, 85, 55, 35, 18, 6]

plt.plot(time_hr, oral_conc, marker="o", color="green", label="Oral")
plt.plot(time_hr, iv_conc, marker="s", color="gold", label="IV")
plt.title("Plasma Concentration-Time Profile: Oral vs IV")
plt.xlabel("Time (hours)"); plt.ylabel("Concentration (ng/mL)")
plt.legend(); plt.show()
```

Note the characteristic shape difference: an oral dose shows an absorption phase (rising concentration) before the elimination (falling) phase, giving a peaked curve, whereas an IV bolus dose starts at its highest concentration (no absorption phase) and falls immediately.

(b) ADR reporting rates across drugs

```
drugs = ["Metformin", "Atenolol", "Amoxicillin", "Diclofenac"]
adr_count = [12, 5, 20, 9]

plt.bar(drugs, adr_count, color="green") # bar chart -- categorical comparison
plt.title("ADR Reports by Drug (2025)")
plt.xlabel("Drug"); plt.ylabel("Number of ADR Reports")
plt.show()
```

(c) Dissolution profiles

```
time_min = [5, 10, 15, 20, 30, 45, 60]
pct_dissolved = [12.5, 28.0, 41.2, 55.8, 74.1, 89.6, 96.3]

plt.plot(time_min, pct_dissolved, marker="o", color="darkgreen")
plt.title("Dissolution Profile")
plt.xlabel("Time (minutes)"); plt.ylabel("% Drug Dissolved")
plt.grid(True); plt.show()
```

5.5 Scientific Interpretation of Plots

Producing a graph is only half of the required skill — this course also expects students to correctly interpret what a plot shows, since this is exactly how a viva or a long-answer question is framed.

- **Concentration-time curve:** peak height (C_{max}) indicates the maximum concentration reached; time to reach it (T_{max}) reflects absorption rate; the area under the curve (AUC) reflects total drug exposure; a steeper decline indicates faster elimination.
- **ADR bar chart:** the tallest bar identifies the drug most frequently associated with adverse reactions in the dataset, prompting closer pharmacovigilance attention.

- **Dissolution profile:** a curve that rises quickly and plateaus near 100% indicates a fast-dissolving formulation; comparing two batches' curves side-by-side reveals batch-to-batch consistency (or lack of it), directly relevant to Quality Control decisions.
- **Box plot:** a narrow box indicates consistent/low-variability results; points plotted outside the 'whiskers' are outliers that may warrant investigation.

Exam Tip

When asked to 'interpret' a graph in an exam, always mention: (1) the overall trend/shape, (2) the specific numeric feature that matters clinically (peak, plateau, outlier), and (3) what that feature means in pharmaceutical/clinical terms — a bare description of the shape without this last step loses marks.

Unit V — Summary

Quick Revision

Matplotlib (import matplotlib.pyplot as plt) turns NumPy/Pandas data into charts. `plt.plot()` draws line plots (best for trends over time, e.g. concentration-time curves); `plt.hist()` draws histograms (distribution of one variable); `plt.scatter()` draws scatter plots (relationship between two variables); `plt.boxplot()` compares distributions/outliers across groups; `plt.bar()` compares categories (e.g. ADR counts by drug). Every plot must be labelled with `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, and `plt.legend()` where multiple series are shown. Interpreting a plot pharmaceutically means connecting its visual features (peak, plateau, outlier, steepness) to a real clinical or quality meaning, not just describing its shape.

Glossary of Key Terms

An alphabetical, consolidated glossary of the important terms used across all five units of this book — ideal for a rapid, last-hour revision before the examination.

Term	Meaning
Algorithm	A step-by-step procedure for solving a problem or completing a task.
Argument	The actual value passed to a function when it is called.
Array (NumPy)	A fast, fixed-type, n-dimensional data structure from the NumPy library used for numeric computation.
Boolean	A data type with only two possible values: True or False.
Break	A statement that immediately exits the enclosing loop.
CSV	Comma-Separated Values — a plain-text file format for tabular data.
DataFrame	A two-dimensional, labelled Pandas data structure resembling a spreadsheet table.
Debugging	The process of finding and fixing errors (bugs) in a program.
Dictionary	An unordered (by position) collection of key-value pairs, written in { }.
Docstring	A string placed as the first statement in a function/module to document its purpose.
Dynamic typing	A language feature where a variable's data type is determined automatically at runtime from its value.
f-string	A formatted string literal (f"...") that lets variable values be embedded directly inside text.
Function	A named, reusable block of code that performs a specific task.
Groupby	A Pandas operation that splits data into groups by column value for separate aggregation.
IDE	Integrated Development Environment — software combining an editor, runner, and debugger.
Immutable	Describes a value that cannot be changed after creation (e.g. a string or tuple).
Index	The numeric position of an element in a sequence, starting at 0.
Interpreter	A program that reads and executes Python code line-by-line.
Iteration	The repeated execution of a block of code, typically via a loop.
Library / Module	A collection of pre-written functions and tools that can be imported into a program.
List	An ordered, mutable collection of items, written in [].
Matplotlib	A third-party Python library for creating charts and graphs.
Mutable	Describes a value that can be changed after creation (e.g. a list or dictionary).
NaN	'Not a Number' — Pandas' representation of a missing numeric value.
NumPy	'Numerical Python' — a third-party library providing fast array-based numeric computation.

Term	Meaning
Pandas	A third-party library built on NumPy for handling structured (tabular) data.
Parameter	The name given to an expected input value in a function's definition.
pip	Python's standard package manager, used to install third-party libraries.
Series	A one-dimensional, labelled Pandas data structure (a single column).
Slicing	Extracting a sub-portion of a sequence using [start:stop:step] notation.
Standard library	A module that ships with Python itself and needs no separate installation.
String	An ordered, immutable sequence of characters representing text.
Syntax	The set of rules that define correctly structured statements in a language.
Third-party library	A library written by the community that must be installed separately (e.g. via pip).
Tuple	An ordered, immutable collection of items, written in ().
Type casting	Explicitly converting a value from one data type to another.
Variable	A named location in memory used to store a value that can change.

Multiple Choice Questions (MCQ Bank)

50 unit-wise multiple choice questions (10 per unit) for quick self-testing. The answer key follows at the end of this section.

Unit I — Introduction to Python Programming

1. Python is best described as a:

(A) Compiled, low-level language (B) High-level, interpreted language (C) Markup language (D) Assembly language

2. Which of the following is a valid Python variable name?

(A) 2dose (B) dose_2 (C) dose-2 (D) dose 2

3. Which function is used to check the data type of a variable?

(A) type() (B) datatype() (C) typeof() (D) check()

4. input() in Python always returns a value of type:

(A) int (B) float (C) str (D) bool

5. Which operator returns the remainder of a division?

(A) // (B) / (C) % (D) **

6. Which of these is NOT a standard library?

(A) math (B) os (C) pandas (D) datetime

7. Which command installs a third-party library?

(A) python install pandas (B) pip install pandas (C) install pandas (D) get pandas

8. The result of "5" + "3" in Python is:

(A) 8 (B) 53 (C) Error (D) 15

9. Which IDE is cell-based, running one block at a time?

(A) PyCharm (B) VS Code (C) Jupyter Notebook (D) Notepad

10. str(25) converts the integer 25 into a:

(A) float (B) string (C) boolean (D) list

Unit II — Control Structures & Functions

1. Which keyword checks additional conditions after an if?

(A) elseif (B) elif (C) else if (D) elseif:

2. Which loop is used when the number of iterations is known in advance?

(A) while (B) for (C) do-while (D) repeat

3. Which statement skips the current iteration and continues the loop?

(A) break (B) continue (C) pass (D) skip

4. Which statement immediately terminates the enclosing loop?

(A) continue (B) pass (C) break (D) exit

5. In a function definition, the value listed is called a:

(A) Argument (B) Parameter (C) Variable (D) Operator

6. What keyword is used to define a function in Python?

(A) function (B) def (C) fun (D) define

7. A function with no explicit return statement returns:

(A) 0 (B) "" (C) None (D) an error

8. Which loop structure uses indentation instead of braces to define its body?

(A) Java loops (B) C loops (C) Python loops (D) None of these

9. What does 'pass' do inside a Python block?

(A) Exits the loop (B) Does nothing (placeholder) (C) Skips the iteration (D) Raises an error

10. Default arguments in a function are specified:

(A) In the function call only (B) In the function definition, using = (C) Using the keyword 'default' (D) They are not supported in Python

Unit III — Data Structures & File Handling

1. Which data structure is immutable?

(A) List (B) Dictionary (C) Tuple (D) Set of lists

2. Dictionary values are accessed using:

(A) Index position (B) Key (C) Slicing only (D) Both index and key

3. What does list[1:3] return?

(A) Elements at index 1 and 3 (B) Elements at index 1 and 2 (C) Elements at index 1, 2, and 3 (D) Element at index 3

4. Which library provides the ndarray object?

(A) Pandas (B) Matplotlib (C) NumPy (D) csv

5. Which module (standard library) is used to read/write CSV files?

(A) pandas (B) csv (C) numpy (D) os

6. Which of these correctly opens a file safely (auto-closing)?

(A) open('f.csv') (B) with open('f.csv') as f: (C) file('f.csv') (D) read('f.csv')

7. csv.DictReader reads each row as a:

(A) List (B) Tuple (C) Dictionary (D) String

8. Negative indexing in Python starts counting from:

(A) The first element (B) The last element (C) Index 0 only (D) It is not supported

9. Which method adds a single item to the end of a list?

(A) list.add() (B) list.append() (C) list.insert(0) (D) list.extend(single_item)

10. A structured healthcare dataset is best described as data organised into:

(A) Random unlabelled text (B) Named columns and rows (C) Images only (D) A single string

Unit IV — Data Handling with Pandas

1. A Pandas Series is:

(A) Two-dimensional (B) One-dimensional, labelled (C) Always numeric only (D) The same as a NumPy array

2. Which function loads a CSV file into a Pandas DataFrame?

(A) pd.load_csv() (B) pd.read_csv() (C) pd.open_csv() (D) pd.import_csv()

3. Which function shows the first 5 rows of a DataFrame by default?

(A) df.top() (B) df.first() (C) df.head() (D) df.show()

4. Which function reports column data types and missing-value counts?

(A) df.describe() (B) df.info() (C) df.summary() (D) df.types()

5. Missing values in Pandas are represented as:

(A) 0 (B) Null (C) NaN (D) None only

6. Which method removes rows containing missing values?

(A) df.fillna() (B) df.dropna() (C) df.remove() (D) df.clean()

7. Which accessor selects data by integer position?

(A) .loc[] (B) .iloc[] (C) .at[] (D) .pos[]

8. groupby() is typically followed by:

(A) A plotting function (B) An aggregation function like mean() (C) A loop only (D) print() only

9. df.describe() by default summarises:

(A) Only text columns (B) Only numeric columns (C) All columns equally (D) Only the index

10. To save a DataFrame without an extra index column, use:

(A) df.to_csv('f.csv') (B) df.to_csv('f.csv', index=False) (C) df.save('f.csv') (D) df.write('f.csv')

Unit V — Data Visualization with Matplotlib

1. Which function creates a line plot in Matplotlib?

(A) plt.line() (B) plt.plot() (C) plt.draw() (D) plt.graph()

2. Which chart type is best for showing the distribution of a single variable?

(A) Scatter plot (B) Histogram (C) Line plot (D) Pie only

3. Which chart type shows the relationship between two numeric variables?

(A) Histogram (B) Box plot (C) Scatter plot (D) Bar chart

4. Which function sets the title of a Matplotlib chart?

(A) plt.name() (B) plt.title() (C) plt.heading() (D) plt.label()

5. Which function displays the legend on a chart?

(A) plt.key() (B) plt.legend() (C) plt.labels() (D) plt.show_legend()

6. A box plot's box represents (approximately):

(A) Minimum and maximum only (B) The interquartile range (25th–75th percentile) (C) Only the mean (D) Only outliers

7. Which import statement is conventionally used for Matplotlib?

(A) import matplotlib as plt (B) import matplotlib.pyplot as plt (C) from matplotlib import * (D) import pyplot

8. An oral drug's concentration-time curve typically shows:

(A) An immediate peak at time 0 (B) A rise (absorption) then a fall (elimination) (C) A constant flat line (D) Only a decline

9. Which function displays the finished plot on screen?

(A) plt.render() (B) plt.display() (C) plt.show() (D) plt.output()

10. A bar chart is most appropriate for comparing:

(A) Two continuous variables (B) Categories, e.g. ADR counts per drug (C) A single variable's distribution (D) Time-series data only

MCQ Answer Key

Unit I — Introduction to Python Programming

1-B 2-B 3-A 4-C 5-C 6-C 7-B 8-B 9-C 10-B

Unit II — Control Structures & Functions

1-B 2-B 3-B 4-C 5-B 6-B 7-C 8-C 9-B 10-B

Unit III — Data Structures & File Handling

1-C 2-B 3-B 4-C 5-B 6-B 7-C 8-B 9-B 10-B

Unit IV — Data Handling with Pandas

1-B 2-B 3-C 4-B 5-C 6-B 7-B 8-B 9-B 10-B

Unit V — Data Visualization with Matplotlib

1-B 2-B 3-C 4-B 5-B 6-B 7-B 8-B 9-C 10-B

Fill in the Blanks

Unit-wise fill-in-the-blank practice with answers given directly below each set — cover the answer and test yourself first.

Unit I

1. Python is a/an _____ language, meaning code is executed line-by-line without a separate compilation step.
2. The function _____ is used to check the data type of a variable.
3. `input()` always returns a value of type _____.
4. _____ is the process of explicitly converting a value from one data type to another.
5. The _____ operator (%) returns the remainder of a division.
6. Libraries that ship with Python and need no installation are called _____ libraries.
7. The command used to install a third-party library is `pip _____ <library-name>`.
8. _____ Notebook allows code to be run cell-by-cell with output shown immediately below.

Answers

1. *interpreted* | 2. *type()* | 3. *string (str)* | 4. *Type casting* | 5. *modulus* | 6. *standard* | 7. *install* | 8. *Jupyter*

Unit II

1. The _____ keyword checks additional conditions after an initial `if`.
2. A _____ loop is preferred when the number of repetitions is known in advance.
3. The _____ statement immediately exits the enclosing loop.
4. The _____ statement skips the rest of the current iteration and moves to the next.
5. In Python, code blocks are defined using _____ rather than curly braces.
6. A function is defined in Python using the keyword _____.
7. The value passed into a function when it is called is known as an _____.
8. A function without an explicit return statement returns _____ by default.

Answers

1. *elif* | 2. *for* | 3. *break* | 4. *continue* | 5. *indentation* | 6. *def* | 7. *argument* | 8. *None*

Unit III

1. A _____ is an ordered, immutable collection written using round brackets.
2. A _____ stores data as key-value pairs written inside curly braces.
3. Python indexing begins at _____.
4. _____ provides the `ndarray` for fast, vectorised numeric computation.
5. The standard library module used to read/write CSV files is _____.
6. `csv._____` reads each row of a CSV file directly as a dictionary.
7. The `with open(...) as ...` syntax automatically _____ the file once the block finishes.

Answers

1. *tuple* | 2. *dictionary* | 3. *0* | 4. *NumPy* | 5. *csv* | 6. *DictReader* | 7. *closes*

Unit IV

1. A Pandas _____ is a one-dimensional labelled array (a single column).
2. A Pandas _____ is a two-dimensional labelled table.
3. `pd._____()` loads a CSV file directly into a DataFrame.
4. Missing numeric values in Pandas are represented as _____.
5. `df._____()` removes rows containing missing values.
6. `.iloc[]` selects data by _____, while `.loc[]` selects data by label.
7. `_____()` splits a DataFrame into groups for separate aggregation, e.g. by drug name.

Answers

1. *Series* | 2. *DataFrame* | 3. *read_csv* | 4. *NaN* | 5. *dropna* | 6. *integer position* | 7. *groupby*

Unit V

1. The conventional import statement for Matplotlib's plotting module is `import matplotlib.pyplot as _____`.
2. `plt._____()` is used to create a line plot.
3. A _____ is used to show the distribution/spread of a single numeric variable.
4. A _____ plot shows the relationship between two numeric variables using individual points.
5. `plt._____()` displays the finished chart on screen.
6. An oral dose's concentration-time curve shows a rise during the _____ phase before falling during elimination.
7. A _____ chart is most appropriate for comparing ADR counts across different drugs.

Answers

1. *plt* | 2. *plot* | 3. *histogram* | 4. *scatter* | 5. *show* | 6. *absorption* | 7. *bar*

Short Answer Questions (2–3 Marks)

Concise model answers, unit-wise — ideal for quick revision and for practising SE-style short questions.

Unit I

Q1. Differentiate between int, float, and str data types with one example each.

int stores whole numbers (age = 45); float stores decimal numbers (weight = 62.5); str stores text in quotes (drug = "Aspirin"). type() confirms each at runtime since Python is dynamically typed.

Q2. What is the difference between a standard library and a third-party library?

A standard library (e.g. math, csv) ships with Python and needs no installation. A third-party library (e.g. NumPy, Pandas) is written by the community and must be installed separately using pip install <name>.

Q3. Why does input() always need type casting before use in a calculation?

input() always returns a string, even if digits were typed. Using it directly in arithmetic raises a TypeError, so it must be wrapped in int() or float() first, e.g. weight = float(input(...)).

Q4. List any four advantages of using an IDE over a plain text editor.

Syntax highlighting, auto-completion/IntelliSense, an integrated debugger to step through code, and immediate detection of syntax errors before running — Jupyter additionally offers cell-based execution.

Unit II

Q1. Differentiate between break and continue.

break exits the enclosing loop immediately, skipping all remaining iterations. continue skips only the rest of the current iteration and proceeds to the next one — the loop itself keeps running.

Q2. Differentiate between a parameter and an argument.

A parameter is the placeholder name listed in a function's definition (def calc(weight):). An argument is the actual value supplied when the function is called (calc(70)).

Q3. What is the role of the return statement in a function?

return sends a value back to the code that called the function and immediately ends the function's execution. A function with no return statement returns None by default.

Q4. Why is indentation important in Python control structures?

Python uses indentation (not braces) to define which statements belong to a given if/for/while/function block. Incorrect or inconsistent indentation causes an IndentationError or changes the program's logic silently.

Unit III

Q1. Differentiate between a list and a tuple.

A list [] is ordered and mutable (items can be changed after creation). A tuple () is ordered but immutable (its contents cannot be modified once created), making it suitable for fixed reference values.

Q2. What is slicing? Give an example.

Slicing extracts a sub-portion of a sequence using `sequence[start:stop:step]`, where `start` is included and `stop` is excluded. Example: `drugs[1:3]` returns items at index 1 and 2.

Q3. What is the purpose of the csv module in Python?

The `csv` module (a standard library) provides functions such as `csv.reader()`, `csv.writer()`, and `csv.DictReader()` to read from and write to CSV files, which are the common export format for hospital/lab data.

Q4. Why are NumPy arrays preferred over plain Python lists for numeric data?

NumPy arrays support vectorised operations (an arithmetic operation applies to every element at once, without a loop), use less memory, and run much faster on large datasets — plus provide built-in statistics like `mean()` and `max()`.

Unit IV

Q1. Differentiate between a Pandas Series and a DataFrame.

A Series is a one-dimensional labelled array (a single column). A DataFrame is a two-dimensional labelled table (multiple rows and columns); every column inside a DataFrame is itself a Series.

Q2. How are missing values identified and handled in Pandas?

`df.isnull().sum()` counts missing (NaN) values per column. They can then be handled by removing affected rows with `df.dropna()`, or by replacing them with `df.fillna()`, e.g. filling with the column's mean.

Q3. Differentiate between `.loc[]` and `.iloc[]`.

`.loc[]` selects rows/columns by their label or name. `.iloc[]` selects rows/columns by their integer position, regardless of the label.

Q4. What does the `groupby()` function do?

`groupby()` splits a DataFrame into groups based on the unique values of a chosen column (e.g. Drug), so an aggregation function such as `mean()` or `count()` can then be applied separately to each group.

Unit V

Q1. Which Matplotlib function is used for each of: trend over time, distribution of one variable, relationship between two variables?

Trend over time → `plt.plot()` (line plot). Distribution of one variable → `plt.hist()` (histogram). Relationship between two variables → `plt.scatter()` (scatter plot).

Q2. List the essential elements of a properly labelled graph.

A descriptive title (`plt.title()`), labelled x- and y-axes with units (`plt.xlabel()`/`plt.ylabel()`), and a legend (`plt.legend()`) when more than one data series is plotted.

Q3. What does a box plot show, and why is it useful for comparing tablet batches?

A box plot shows the minimum, first quartile, median, third quartile, maximum, and outliers of a dataset. Comparing box plots across batches quickly reveals differences in spread/consistency and highlights outlying results.

Q4. Why does an oral dose's concentration-time curve differ in shape from an IV bolus dose's curve?

An oral dose must first be absorbed, so concentration rises (absorption phase) before falling (elimination phase), producing a peaked curve. An IV bolus enters the bloodstream directly at its highest concentration, so the curve starts at its peak and falls immediately, with no absorption phase.

Long Answer Questions (10 Marks) — Full Detailed Answers

These are full-length, exam-ready model answers written to ESE (End Semester Examination) depth. Use them both to learn the concepts in complete detail and as a template for how to structure your own long answers under exam conditions.

Unit I — Introduction to Python Programming

Q1. Explain the concept of variables and data types in Python. Describe the built-in data types with suitable pharmaceutical examples, and explain how type casting is performed.

A variable in Python is a named location in computer memory used to store a value that may change as a program runs. Unlike languages such as C or Java, Python does not require a variable's type to be declared in advance — the interpreter automatically determines the type from the value assigned to it at the moment of assignment. This behaviour is called dynamic typing, and it is one of the reasons Python is considered beginner-friendly: a student can write `patient_age = 45` and Python automatically treats `patient_age` as an integer, without any separate type declaration.

Python provides several built-in data types relevant to this course. The `int` type stores whole numbers with no decimal component, such as a patient's age (`age = 45`) or a tablet count (`tablet_count = 30`). The `float` type stores numbers with a decimal component, essential for measurements such as body weight (`weight_kg = 62.5`) or a calculated dose. The `str` (string) type stores text enclosed in single or double quotes, used for drug names, patient names, and batch codes (`drug_name = "Paracetamol"`). The `bool` (Boolean) type stores only one of two values, `True` or `False`, and is produced naturally by comparison expressions such as `is_diabetic = (blood_sugar > 126)`. A less commonly used complex type stores numbers with real and imaginary parts, rarely required in pharmaceutical programming.

The type of any variable can be checked at any point using the built-in `type()` function, for example `print(type(weight_kg))` will output `<class 'float'>`. This function is frequently used both for learning and for debugging, since a program that unexpectedly fails often does so because a value has an unexpected type.

Type casting (also called type conversion) is the explicit process of converting a value from one data type to another, performed using Python's built-in conversion functions: `int()`, `float()`, `str()`, and `bool()`. This is essential in practice because values read using `input()` are always returned as strings, even when the user types a number. For example, `age = input("Enter age: ")` stores whatever was typed as a string; to use it in a calculation it must first be converted: `age = int(input("Enter age: "))`. Similarly, if a numeric result needs to be joined with a text message, the number must first be converted to a string using `str()`, since Python does not automatically convert types when combining a string and a number with the `+` operator — attempting `"Dose: " + 500` directly raises a `TypeError`, whereas `"Dose: " + str(500)` succeeds and produces `"Dose: 500"`.

In summary, understanding variables and data types — and knowing precisely when and how to cast between them — is the foundation on which every later unit of this course is built, since incorrect or missing type casting is the single most common source of runtime errors in beginner Python programs, particularly those that take user input for pharmaceutical calculations such as dosage or BMI.

Q2. What is an IDE? Explain its advantages over a plain text editor, and describe the features of any two IDEs commonly used for Python programming.

An Integrated Development Environment (IDE) is a software application that combines several tools needed for writing, testing, and debugging code into a single, unified interface: a code editor, a way to run the program, and typically a debugger. This is in contrast to writing code in a plain text editor (such as Notepad), which offers no assistance beyond storing the typed characters, and then running the file separately from a command line.

IDEs offer several concrete advantages that matter especially to a beginner working with pharmaceutical calculations, where a silent logical error could translate into an incorrectly computed dose. First, syntax highlighting displays keywords, strings, comments, and variable names in different colours, making it visually easy to notice a missing quotation mark or misspelled keyword. Second, auto-completion (sometimes called IntelliSense) suggests variable and function names as you type, both saving time and reducing typing errors such as misspelling `weight_kg` as `wieght_kg`. Third, an integrated debugger allows the programmer to pause execution at a chosen line (a breakpoint), inspect the current value of every variable, and step through the program one line at a time to find exactly where a calculation goes wrong. Fourth, most IDEs underline or flag syntax errors immediately as code is typed, before the program is even run, catching mistakes far earlier than a text editor ever could. Finally, an integrated terminal lets libraries be installed and scripts run without leaving the application.

Jupyter Notebook is especially suited to the data-analysis-heavy later units of this course (Units III–V), because it organises a program into individual cells that can each be run independently, with the output displayed immediately below that cell. This means a student can load a dataset in one cell, inspect it in the next, and try different plotting options in a third, re-running only the cell that changed rather than the entire script — extremely useful when exploring a pharmaceutical dataset interactively.

PyCharm, by contrast, is a full-featured, professional-grade IDE better suited to larger, multi-file projects. It provides powerful project-wide search and refactoring tools (e.g. renaming a function consistently everywhere it is used), sophisticated debugging, and built-in management of virtual environments (isolated sets of installed libraries for different projects). VS Code (Visual Studio Code) sits between these two: a free, lightweight, highly extensible editor that becomes a capable Python IDE once the official Python extension is installed, popular for its speed and flexibility across many programming languages, not only Python.

In conclusion, while any of these IDEs can run identical Python code, the choice depends on the nature of the task: Jupyter Notebook for exploratory, cell-based data analysis (most relevant to Units III–V of this course), and PyCharm or VS Code for writing and organising larger, structured Python programs such as the dosage- and BMI-calculation applications introduced in Unit II.

Unit II — Control Structures & Functions

Q3. Write a Python program to calculate BMI and classify it according to WHO categories, using proper functions. Explain the logic and control structures used.

Body Mass Index (BMI) is a widely used screening measure calculated as a person's weight in kilograms divided by the square of their height in metres: $\text{BMI} = \text{weight (kg)} \div [\text{height (m)}]^2$. The World Health Organization classifies the resulting value into four broad categories: Underweight (BMI below 18.5), Normal weight (18.5 up to 25), Overweight (25 up to 30), and Obese (30 and above). A well-structured Python program for this task should separate the calculation itself from the classification logic, using two distinct functions, and should take user input safely with correct type casting.

```
def calculate_bmi(weight_kg, height_m):
    """Return BMI given weight (kg) and height (m)."""
    return weight_kg / (height_m ** 2)

def classify_bmi(bmi):
    """Return the WHO BMI category as a string."""
    if bmi < 18.5:
        return "Underweight"
    elif bmi < 25:
        return "Normal weight"
    elif bmi < 30:
        return "Overweight"
    else:
        return "Obese"

def main():
    weight = float(input("Enter weight (kg): "))
    height = float(input("Enter height (m): "))
    bmi = round(calculate_bmi(weight, height), 2)
    category = classify_bmi(bmi)
    print(f"BMI: {bmi} -> Category: {category}")

main()
```

The program's logic proceeds in three clear stages. First, `calculate_bmi()` performs only the arithmetic — division of weight by the square of height — and returns a single numeric value, kept deliberately separate from any decision-making so that it can be reused or tested independently. Second, `classify_bmi()` applies an if-elif-else control structure to that numeric value. Because Python evaluates an if-elif-else chain from top to bottom and stops at the first True condition, each elif only needs to state the upper boundary of its range (`bmi < 25` correctly means '18.5 and above, but below 25', because any value below 18.5 would already have been caught by the first if). This is a subtlety that is frequently tested, since writing `bmi < 25` as a stand-alone condition without the chain structure would incorrectly classify an underweight person as Normal weight too. Third, the `main()` function acts as the program's driver: it collects input from the user, casts it immediately to `float()` (since `input()` always returns a string and the calculation would otherwise fail), calls the two functions in sequence, and rounds the final BMI to two decimal places using `round()` purely for readable display.

This structure — small, single-purpose functions combined through one driver function — is what the syllabus refers to as writing 'modular programs for pharmaceutical applications', and is directly transferable to the parallel dosage-calculation program, which follows the identical pattern: one function to compute a numeric result, one function to classify or interpret it, and a `main()` function that ties the whole workflow together.

Q4. Differentiate between for loops and while loops with examples, and explain the purpose of break, continue, and pass statements.

Python provides two loop constructs, both used to repeat a block of code, but suited to different situations. A for loop iterates over the items of a known, finite sequence — a list, a string, or a range of numbers generated by `range()` — running the loop body exactly once for each item in that sequence. It is the natural choice whenever the number of repetitions is already known or is tied to the size of a collection, for example

printing one line for each of a fixed list of drug doses: `for dose in doses_mg: print(dose)`. A while loop, in contrast, repeats its body only as long as a specified condition remains True, and is used when the number of repetitions cannot be known in advance but instead depends on some changing state — for example continuing to dispense stock until it runs out: `while stock > 0: stock -= 10`.

A critical practical difference is that a while loop requires the programmer to manually update the variable being tested inside the loop body; forgetting to do so (e.g. forgetting `stock -= 10`) produces an infinite loop that never terminates, since the condition never becomes False. A for loop carries no such risk, since Python itself advances through the sequence automatically.

Three additional statements refine control over loop execution. The `break` statement immediately terminates the entire enclosing loop the moment it is encountered, skipping any remaining iterations altogether — useful, for instance, when searching a list of batch numbers and stopping as soon as the required batch is found, since continuing to check the rest of the list would be wasted effort. The `continue` statement is less drastic: it skips only the remainder of the current iteration and proceeds directly to the next one, without exiting the loop itself — useful when processing a list of readings that contains some invalid placeholder values (e.g. -1 for a missing reading), allowing those entries to be silently skipped while every other reading is still processed normally. The `pass` statement, finally, performs no action whatsoever; it exists purely because Python's syntax requires every block (if, for, while, function, or class) to contain at least one statement, so `pass` is used as a temporary or intentional placeholder where a block is syntactically required but no code is needed yet, for example while a function's logic is still being planned.

In summary: choose a for loop when repetitions are tied to a known sequence, a while loop when repetitions depend on a condition that may not be known in advance; use `break` to exit a loop early once its purpose has been fulfilled, `continue` to skip an individual unwanted iteration without ending the loop, and `pass` wherever a syntactically required block is deliberately left empty.

Unit III — Data Structures & File Handling

Q5. Differentiate between lists, tuples, and dictionaries with suitable examples, and explain how NumPy arrays differ from ordinary Python lists.

Python provides three built-in data structures that are central to organising more than a single value under one variable name, and each is suited to a different situation. A list, written using square brackets `[ ]`, is an ordered and mutable collection — 'ordered' meaning each item has a fixed position (index) that can be relied upon, and 'mutable' meaning items can be added, removed, or changed after the list is created. A list of drug names, `drug_names = ["Paracetamol", "Ibuprofen", "Amoxicillin"]`, can later be modified with `drug_names.append("Cetirizine")` or `drug_names[0] = "Paracetamol 500mg"`.

A tuple, written using round brackets `( )`, is also ordered but is immutable: once created, its contents cannot be altered — attempting to reassign an element raises a `TypeError`. This makes tuples the appropriate choice for values that should be protected from accidental modification, such as a fixed reference range: `normal_bp_range = (90, 120)`. A dictionary, written using curly braces `{ }`, stores data as key-value pairs, where each value is retrieved not by a numeric position but by a unique, meaningful key. This is the natural structure for a single structured record, such as one patient's data: `patient_record = {"name": "Rahul Verma", "age": 34, "weight_kg": 68.5}`, where `patient_record["age"]` retrieves 34 directly by name rather than by having to remember its numeric position.

The choice between these three, therefore, depends on the shape of the data: use a list for an ordered collection of similar items that may change over time (a batch's set of quantities dispensed), a tuple for a small, fixed set of values that must never change (a reference range or a coordinate pair), and a dictionary whenever data is more naturally described by named fields than by position (a patient's or drug's full record).

NumPy arrays (the `ndarray` object, from the third-party NumPy library) differ from ordinary Python lists in several important respects, even though both can superficially hold a sequence of numbers. First, a NumPy array requires every element to be of the same data type (usually all floats or all integers), whereas a Python list may freely mix types — this restriction is exactly what allows NumPy to store data far more compactly in memory and to process it far faster. Second, NumPy supports vectorised arithmetic: an operation such as `percent_dissolved * 2` or `dissolution_times + 5` applies to every element of the array simultaneously in a single line, with no explicit loop required, whereas achieving the same result with a plain Python list would require writing a for loop to process each element individually. Third, NumPy provides a rich set of ready-made statistical functions directly as array methods — `array.mean()`, `array.max()`, `array.std()` — that a plain list does not offer natively. For these reasons, NumPy arrays (and, building on them, Pandas Series and DataFrames in Unit IV) are the preferred data structure whenever a pharmaceutical dataset involves numeric computation at any meaningful scale, while plain lists, tuples, and dictionaries remain the right tool for smaller, mixed-type, or record-like data.

Unit IV — Data Handling with Pandas

Q6. Explain the Pandas Series and DataFrame structures. Describe, with examples, how a dataset is inspected and how missing values are handled.

Pandas is a third-party Python library, built on top of NumPy, specifically designed to work with structured (tabular) data of the kind produced by hospitals, laboratories, and clinical research — patient records, prescription logs, ADR reports, and pharmacokinetic (PK) study datasets. It introduces two core data structures. A Series is a one-dimensional, labelled array: essentially a single column of values, each carrying an associated index label, created for example with `ages = pd.Series([45, 62, 34, 58], name="Age")`. A DataFrame is a two-dimensional, labelled table — conceptually identical to a spreadsheet, with named columns and an index of rows — and every individual column inside a DataFrame is itself, internally, a Series. A DataFrame can be built directly from a Python dictionary of equal-length lists, `df = pd.DataFrame({"PatientID": [...], "Age": [...], "Drug": [...]})`, or, far more commonly in practice, loaded directly from an existing file using `pd.read_csv("file.csv")` or `pd.read_excel("file.xlsx")`.

Before any analysis is attempted, a newly loaded dataset must always be inspected first, and Pandas provides four essential functions for this. `df.head()` (and the equivalent `df.tail()`) displays the first (or last) few rows, giving an immediate visual sense of the data's shape and content. `df.info()` reports, column by column, the data type stored and — critically — the count of non-null (present) values, immediately revealing which columns contain missing data. `df.describe()` computes summary statistics — count, mean, standard deviation, minimum, the 25th/50th/75th percentiles, and maximum — for every numeric column at once, giving a fast statistical overview without writing any custom code. Together, these four functions should be run as a matter of routine on any dataset before proceeding further, since attempting to analyse or plot a dataset without first understanding its shape and completeness is a common source of errors.

Real pharmaceutical and clinical datasets are rarely complete: a patient's weight may not have been recorded, or a PK sampling time-point may have been missed. Pandas represents any such missing numeric

value internally as NaN ('Not a Number'). The extent of missingness in a dataset is checked with `df.isnull().sum()`, which returns the count of missing values for every column. Two main strategies exist to handle them. `df.dropna()` removes every row that contains at least one missing value — appropriate when a record is simply unusable without that particular field, such as a PK sample with no recorded collection time. Alternatively, `df.fillna(value)` replaces missing values with a specified value, most commonly a sensible statistical estimate such as the column's own mean or median, for example `df["Weight_kg"].fillna(df["Weight_kg"].mean())` — appropriate for a physiological measurement where a reasonable estimate is preferable to discarding the whole record, but a decision that must always be made carefully and never used to invent a missing safety or efficacy outcome. Choosing correctly between these two approaches, and justifying that choice, is an important applied skill this unit specifically develops.

Unit V — Data Visualization with Matplotlib

Q7. Explain how Matplotlib is used to plot and interpret a drug's concentration-time profile and a dissolution profile. Why is proper labelling of a graph important?

Matplotlib is Python's foundational third-party library for turning numeric data into visual charts, accessed through its pyplot module, conventionally imported as `import matplotlib.pyplot as plt`. Two of the pharmaceutically important plots this unit builds towards are the concentration-time profile and the dissolution profile, and both are naturally drawn as line plots, since each represents how one quantity changes continuously over time.

A concentration-time profile plots the measured plasma drug concentration on the y-axis against sampling time on the x-axis, typically for both an oral and an intravenous (IV) route of administration on the same axes for comparison: `plt.plot(time_hr, oral_conc, marker="o", label="Oral")` followed by `plt.plot(time_hr, iv_conc, marker="s", label="IV")`. The two curves have characteristically different shapes: an orally administered dose must first be absorbed into the bloodstream, so its concentration rises during an initial absorption phase before falling during the elimination phase, producing a peaked curve; an IV bolus dose, by contrast, enters the bloodstream directly and instantaneously, so its concentration is highest at time zero and falls immediately, with no absorption phase at all. Scientifically interpreting this plot means identifying the peak concentration (C_{max}) and the time at which it occurs (T_{max}) for the oral curve — both of which describe the rate and extent of absorption — and noting how steeply each curve declines, which reflects the rate of drug elimination.

A dissolution profile similarly plots the percentage of drug dissolved (y-axis) against time in minutes (x-axis), typically for a solid oral dosage form under standard dissolution-testing conditions: `plt.plot(time_min, pct_dissolved, marker="o")`. A curve that rises quickly and plateaus near 100% indicates a fast-dissolving, readily bioavailable formulation, whereas a slower or incomplete rise may indicate a formulation or Quality Control issue. Plotting two batches' dissolution curves on the same axes, or comparing them with a box plot at a single time-point, allows batch-to-batch consistency to be assessed visually — a routine Quality Control task.

Proper labelling is not a cosmetic addition but a scientific requirement of any chart presented in a pharmaceutical context. `plt.title()` states what the chart shows; `plt.xlabel()` and `plt.ylabel()` state exactly what each axis measures together with its unit (e.g. 'Time (hours)', 'Concentration (ng/mL)') — without units, a reader cannot correctly interpret the numeric scale at all; and `plt.legend()` is essential whenever more than one data series (such as Oral vs IV) is plotted on the same axes, since without it there is no way to tell which

line corresponds to which condition. An unlabelled graph, however visually correct, would be considered scientifically incomplete and would lose marks in both a practical examination and, more importantly, in any real regulatory or research report, where an unlabelled chart could be misread and lead to an incorrect clinical or Quality Control conclusion.

Practice / Lab Programs

Ten complete practice programs spanning all five units, suitable for practical file submission and lab-record preparation.

Program 1: Write a program to input a patient's weight and a dose per kg, and print the total calculated dose.

```
weight = float(input("Enter weight (kg): "))
dose_per_kg = float(input("Enter dose (mg/kg): "))
total_dose = weight * dose_per_kg
print(f"Total dose: {total_dose} mg")
```

Program 2: Write a program to classify a patient's BMI into WHO categories using if-elif-else.

```
weight = float(input("Weight (kg): "))
height = float(input("Height (m): "))
bmi = weight / (height ** 2)
if bmi < 18.5:
    print("Underweight")
elif bmi < 25:
    print("Normal weight")
elif bmi < 30:
    print("Overweight")
else:
    print("Obese")
```

Program 3: Write a function-based program that returns True if a patient needs renal dose adjustment (age > 65 and creatinine > 1.5).

```
def needs_adjustment(age, creatinine):
    return age > 65 and creatinine > 1.5

print(needs_adjustment(72, 1.8))    # True
print(needs_adjustment(40, 1.0))    # False
```

Program 4: Write a program using a for loop to print a 7-day dosing schedule.

```
for day in range(1, 8):
    print(f"Day {day}: take 1 tablet after food")
```

Program 5: Write a program to store 3 patients' records in a list of dictionaries and print each patient's name and age.

```
patients = [
    {"name": "Anita", "age": 45},
    {"name": "Rahul", "age": 34},
    {"name": "Sunita", "age": 58}
]
for p in patients:
```

```
print(p["name"], "-", p["age"])
```

Program 6: Write a program using NumPy to compute the mean and maximum of a set of dissolution readings.

```
import numpy as np
readings = np.array([12.5, 28.0, 41.2, 55.8, 74.1, 89.6, 96.3])
print("Mean:", readings.mean())
print("Max:", readings.max())
```

Program 7: Write a program to read a CSV file of patients and print only those older than 60.

```
import pandas as pd
df = pd.read_csv("patients.csv")
elderly = df[df["Age"] > 60]
print(elderly)
```

Program 8: Write a program to load an ADR dataset and count reports grouped by severity.

```
import pandas as pd
adr = pd.read_csv("adr_reports.csv")
print(adr.groupby("Severity")["Drug"].count())
```

Program 9: Write a program to plot a labelled concentration-time curve.

```
import matplotlib.pyplot as plt
time_hr = [0, 1, 2, 4, 6, 8]
conc = [0, 78, 92, 65, 40, 22]
plt.plot(time_hr, conc, marker="o", color="green")
plt.title("Concentration-Time Curve")
plt.xlabel("Time (hr)"); plt.ylabel("Conc (ng/mL)")
plt.show()
```

Program 10: Write a program to plot a bar chart of ADR counts per drug.

```
import matplotlib.pyplot as plt
drugs = ["Metformin", "Atenolol", "Amoxicillin"]
counts = [12, 5, 20]
plt.bar(drugs, counts, color="gold")
plt.title("ADR Reports by Drug")
plt.show()
```

Model Question Paper

B.Pharm — Semester I

BP101T: Basics of Python Programming for Pharmaceutical Sciences

Time: 2 Hours

Maximum Marks: 30 (ESE)

Note: Attempt all questions from Section A. Attempt any four from Section B. Attempt any two from Section C.

Section A — Very Short Answer (1 mark each; 5 marks)

6. What is the return type of Python's input() function?
7. Name the third-party library used for data visualization in this course.
8. Which keyword defines a function in Python?
9. What symbol is used to write a dictionary in Python?
10. Name any one function used to inspect a Pandas DataFrame.

Section B — Short Answer (5 marks each; any 4 = 20)

11. Differentiate between a list, a tuple, and a dictionary with examples.
12. Explain the difference between break and continue with a suitable example.
13. Differentiate between a Pandas Series and a DataFrame.
14. Explain how missing values are detected and handled in a Pandas DataFrame.
15. List and briefly explain any four types of plots available in Matplotlib.
16. Explain type casting in Python with two examples.

Section C — Long Answer (10 marks each; any 2 = 20)

17. Write a complete, function-based Python program to calculate BMI and classify it according to WHO categories. Explain the control structures used.
18. Explain the Pandas Series and DataFrame structures, and describe with examples how a healthcare dataset is inspected and cleaned of missing values.
19. Explain, with labelled code examples, how Matplotlib is used to plot and scientifically interpret a drug's concentration-time profile for oral and IV administration.
20. Differentiate between for loops and while loops, and explain the purpose of break, continue, and pass with suitable pharmaceutical examples.

Note

This model paper is constructed by DRx Learning Hub to match the syllabus's stated marks distribution (Max. Marks 50 = Sessional 20 + ESE 30) and question style; the actual university paper format may vary slightly — always cross-check against your university's latest question paper pattern.

Recommended References

(Preferably latest editions, as listed in the official syllabus)

21. Weiss, C.J., 2017. Scientific Computing for Chemists with Python. Available at: weisscharlesj.github.io/SciCompforChemists
22. Perkovic, L., 2015. Introduction to Computing Using Python: An Application Development Focus, 2nd ed. Wiley.
23. Sweigart, A., 2025. Automate the Boring Stuff with Python, 3rd ed. Available at: automatetheboringstuff.com
24. W3Schools, n.d. Python Tutorial. Available at: w3schools.com/python
25. Healthcare datasets for education and practice: Kaggle (healthcare records), government portals (data.gov.in, healthdata.gov, WHO), and clinical trial registries (ctri.nic.in, clinicaltrials.gov) — always use data responsibly.

About DRx Learning Hub

DRx Learning Hub, compiled and edited by DRx Deepak Jatav, produces exam-ready, syllabus-aligned study material for B.Pharm students, with a focus on the NEP 2020 curriculum. This complete e-book for BP101T brings together all five units of the official syllabus into a single, long-form, unit-wise resource — combining detailed theory, pharma-context worked examples, and a full self-testing practice section — for use ahead of Sessional and End Semester Examinations.

— End of E-Book —